

C# / .NET Fresher Training Documentation

Day 1 – .NET & C# Fundamentals

What is .NET?

.NET is a **developer platform** created by Microsoft to build: - Web Applications (ASP.NET / ASP.NET Core) - APIs & Microservices - Desktop applications - Cloud-based systems

Key Components of .NET

- **CLR (Common Language Runtime)** – Executes code
- **BCL (Base Class Library)** – Predefined classes
- **JIT Compiler** – Converts IL to machine code

Execution Flow

C# Code → IL Code → JIT → Machine Code

Data Types

Value Types

- int, double, bool, struct
- Stored in **stack**

Reference Types

- class, string, array
- Stored in **heap**

```
int a = 10;           // Value type
string name = "John"; // Reference type
```

Coding Best Practices

- Avoid magic numbers
- Use constants
- Write small methods

OOPS Concepts (Industry Level)

Why OOPS?

- Code reusability
- Maintainability
- Scalability

Encapsulation

Protecting data using access modifiers.

```
class BankAccount
{
    private double balance;

    public void Deposit(double amount)
    {
        if (amount > 0)
            balance += amount;
    }

    public double GetBalance()
    {
        return balance;
    }
}
```

Why important? - Prevents misuse of data - Controls access

Inheritance

Reusing existing code.

```
class Employee
{
    public int Id { get; set; }
    public string Name { get; set; }
}

class Manager : Employee
{
    public int TeamSize { get; set; }
}
```

Use Case: Common properties shared across classes

Polymorphism

One interface, multiple implementations.

```
class Notification
{
    public virtual void Send()
```

```

    {
        Console.WriteLine("Sending notification");
    }
}

class EmailNotification : Notification
{
    public override void Send()
    {
        Console.WriteLine("Sending Email");
    }
}

```

Abstraction

Expose only what is required.

```

interface IPayment
{
    void Pay();
}

class CreditCardPayment : IPayment
{
    public void Pay()
    {
        Console.WriteLine("Paid using credit card");
    }
}

```

Models & DTOs

Model

Represents application data.

```

class ProductModel
{
    public int Id { get; set; }
    public string Name { get; set; }
    public decimal Price { get; set; }
}

```

DTO (Data Transfer Object)

Used to transfer data between layers.

```
class ProductDto
{
    public string Name { get; set; }
    public decimal Price { get; set; }
}
```

Why separate? - Security - Performance

Collections (Deep Understanding)

Non-Generic Collections (Avoid in modern apps)

```
ArrayList list = new ArrayList();
list.Add(1);
list.Add("Test");
```

✗ No type safety ✗ Boxing/Unboxing

Generic Collections

List

```
List<string> names = new List<string>();
names.Add("Amit");
names.Add("Rahul");
```

✓ Fast ✓ Type-safe

Dictionary<TKey, TValue>

```
Dictionary<int, string> users = new Dictionary<int, string>();
users[1] = "Admin";
```

When to use: - Lookup operations - Cache

Exception Handling (Professional Way)

Why handle exceptions?

- Prevent crashes
- Log errors

```
try
{
    int result = int.Parse("abc");
}
catch (FormatException ex)
```

```
{  
    Console.WriteLine(ex.Message);  
}  
catch (Exception ex)  
{  
    Console.WriteLine("Unknown error");  
}
```

Custom Exception

```
class BusinessException : Exception  
{  
    public BusinessException(string message) : base(message) {}  
}
```

Delegates & Events (Real Use)

Delegate

Represents method reference.

```
delegate int Calculate(int a, int b);
```

```
int Add(int x, int y) => x + y;
```

```
Calculate calc = Add;  
Console.WriteLine(calc(2,3));
```

Events

Used for loose coupling.

```
class Order  
{  
    public event Action OrderPlaced;  
  
    public void PlaceOrder()  
    {  
        OrderPlaced?.Invoke();  
    }  
}
```

Filters (ASP.NET Core)

Types of Filters

- Authorization
- Action
- Exception

```
public class ExceptionFilter : IExceptionHandler
{
    public void OnException(ExceptionContext context)
    {
        Console.WriteLine(context.Exception.Message);
    }
}
```

Performance & Optimization

Key Rules

- Avoid blocking calls
- Use async/await
- Reduce memory allocation

```
public async Task<string> GetDataAsync()
{
    await Task.Delay(1000);
    return "Done";
}
```

Clean Code & SOLID Basics

- Single Responsibility Principle
- Open/Closed Principle
- Dependency Injection

```
interface ILogger
{
    void Log(string message);
}
```

Real Project Practices

Layered Architecture

- Controller

- Service
- Repository

Common Mistakes by Freshers

- Hard coding values
- No validation
- No exception handling

Final Advice

✓ Write readable code ✓ Think before optimizing ✓ Learn debugging ✓ Follow standards

End of Document